

Wallet Pools

12 min read

Wallet Pools let you group multiple vault accounts under a single logical source and use that group to send transactions. Each Wallet Pool is a protected tag of a new type, giving it all the approval and governance controls that protected tags carry, plus the ability to route transactions across the vault accounts it contains. Instead of routing all withdrawals through one vault account, Fireblocks distributes transactions across the vault accounts in your pool, selecting a healthy vault account for each transaction.

This is especially useful for high-volume operations such as retail withdrawals, where a single vault account can become a bottleneck under load.

Why use Wallet Pools

When a single vault account handles a high volume of outgoing transactions, it can run into nonce congestion on EVM chains. A stuck transaction blocks every subsequent transaction from that vault account until it resolves, which can cascade into significant delays.

Wallet Pools address this by:

- Distributing transactions across multiple vault accounts so no single vault account is overloaded
- Routing around congested vault accounts automatically
- Improving privacy by rotating across multiple on-chain addresses rather than always using the same wallet
- Eliminating the need to build custom vault rotation logic in your own infrastructure

Use cases

A Wallet Pool is a mechanism that solves the same congestion problem in three different places. In all three cases, you set the pool up the same way, then point the relevant flow at it instead of at a single vault account.

- **Withdrawal vaults:** High-volume withdrawals run through a small set of hot vault accounts, and on EVM chains one stuck transaction blocks every later withdrawal from that vault account until it clears. Routing withdrawals through a pool spreads them across several vault accounts, skips the congested ones, and rotates across multiple on-chain addresses so your withdrawal pattern is less predictable. Set the pool as the transaction source, as described in Step 4 below.
- **Gas Station:** Gas Station fuels every vault account in your workspace from a single gas tank, so on EVM, all fueling shares one sequential nonce stream. That stream stalls exactly when

you can least afford it, during a traffic spike when many vault accounts need topping up at once. Using a pool as the gas tank spreads the fueling across several vault accounts. In

Settings > Gas station tank, select either a single vault account or a Wallet Pool.

- **Gasless relay:** With gasless transactions, your end users hold no native asset, and a relayer vault account signs and pays gas on their behalf. That relayer is one vault account with one nonce stream, so it becomes the bottleneck as gasless volume grows. Using a pool as the relayer source spreads relaying across several vault accounts. In **Settings > Initiate gasless transactions > EVM**, the relayer source selector lets you choose either a single vault account or a Wallet Pool.

Four steps to route through a pool

Getting a pool into production requires four steps. Steps 1 and 2 set up the pool, step 3 makes sure your Policy will let it send, and step 4 is the transaction itself.

Step 1: Create a Wallet Pool tag

A Wallet Pool is a protected tag whose type is `WALLET_POOL`. Creating one requires an Owner, Admin, Non-Signing Admin, or Editor role, and no quorum approval.

In the Console, from the Tags page

1. Navigate to **Utilities > Tags** (<https://console.fireblocks.io/v2/tags>) (**Settings > Tags** (<https://console.fireblocks.io/v2/settings/tags>) in some workspaces).
2. Select **Create Tag**.
3. Enter a name, optional description, and color for the tag.
4. Under tag type, select **Wallet Pool**.
5. Select **Create**.

In the Console, from the vault account view

1. Navigate to **Accounts > Vault** (<https://console.fireblocks.io/v2/accounts/vault>) and find the vault account you want to add to the Wallet Pool.
2. In the **Tags** column, select **+**.
3. Enter a name for your Wallet Pool tag, then select **Create [name] tag**.
4. Under tag type, select **Wallet Pool**.
5. Enter an optional description and select a color for the Wallet Pool tag.
6. Select **Create tag**.

You can also create a pool from the **Wallet Pools** widget in **Accounts**.

Over the API: POST `/v1/tags`

```
{
  "label": "Withdrawal Vaults",
```

Copy

```

    "description": "Withdrawal pool",
    "type": "WALLET_POOL"
  }

```

The response returns the tag, and its `id` is your pool ID. Keep it, as every later step refers to it.

Pool names must be unique across your workspace. A name already in use by a protected tag is unavailable for a pool, and vice versa.

Step 2: Add vault accounts to the pool

Attaching vault accounts turns the tag into a routable pool. This step is a two-part flow. The Owner, Admin, Non-Signing Admin, and Editor roles can all submit a request to add or remove vault accounts. Approving that request requires the Owner, Admin, or Non-Signing Admin role; an Editor can submit attachments and detachments but cannot approve them.

In the Console, from the vault account view

1. Navigate to [Accounts > Vault](https://console.fireblocks.io/v2/accounts/vault) (<https://console.fireblocks.io/v2/accounts/vault>).
2. Find the vault account you want to add to the pool, then select **+** in the **Tags** column.
3. Select the Wallet Pool tag you want to use for the vault account.
4. Select **Apply**.

In the Console, from the Tags page

1. Navigate to [Utilities > Tags](https://console.fireblocks.io/v2/tags) (<https://console.fireblocks.io/v2/tags>) ([Settings > Tags](https://console.fireblocks.io/v2/settings/tags) (<https://console.fireblocks.io/v2/settings/tags>) in some workspaces).
2. Locate the Wallet Pool tag and select **+** in the **Attachments** column.
3. Select the vault accounts you want to add.
4. Submit for approval.

Over the API: **POST** `/v1/vault/accounts/attached_tags`

```

{
  "vaultAccountIds": ["12", "13", "14"],
  "tagIdsToAttach": ["<poolId>"]
}

```

Copy

Since a pool is a protected tag, the response returns these under `pendingOperations`, not `appliedOperations`. Each pending entry carries an `approvalRequestId`. Poll `GET /v1/tags/approval_requests/{id}` to track it, and expect the vault accounts to start receiving traffic only once approval completes. Approvers act in the Fireblocks mobile app or under **Pending Approvals** in the Console.

Removing vault accounts later

Detaching follows the same two-part flow: an Owner, Admin, Non-Signing Admin, or Editor submits the request, and an Owner, Admin, or Non-Signing Admin approves it. In the Console, either navigate to **Accounts** > **Vault**, hover your pointer over the Wallet Pool tag on the vault account and select **X**; or, navigate to **Utilities** > **Tags**, select **+** in the pool's **Attachments** column, and under Attached entities select the **X** on a vault account. Over the API, call `POST /v1/vault/accounts/attached_tags` with `tagIdsToDetach`.

Step 3: Verify your Policy allows the pool to send

Do not skip this step. It is the most common reason a first pool transaction fails.

Fireblocks resolves a pool transaction to one of the pool's member vault accounts, and your Policy is then evaluated against that resolved vault account. When your existing rules only permit specific vault account IDs as sources, a pool transaction that resolves to any other member is blocked, even though the pool itself is configured correctly. Since routing deliberately moves between members, a rule that works on one transaction can block the next.

The fix is to write the rule against the pool rather than against individual vault accounts. Add or amend a rule whose source is the Wallet Pool and whose destinations cover everywhere you intend to send.

In the Console: navigate to **Policies**, add or edit a rule, and under source select the pool from the **Wallet Pools** category. Set the destinations to the addresses or accounts the pool needs to reach.

Referencing the pool rather than individual vault accounts also keeps the rule stable as your vault inventory grows. Add or remove vault accounts from the pool through the approval flow, and both routing and policy enforcement follow automatically, with no policy edit required.

Step 4: Send with the pool as the source

In the Console: navigate to **Accounts** > **Vault** and select **Withdraw** next to the pool in the **Wallet Pools** widget. You can also open the transfer dialog and select the pool under **Wallet Pools** in the source list.

Over the API: send a `POST` request to `/v1/transactions` with the source type set to `WALLET_POOL` and the pool's tag ID:

```
{
  "assetId": "ETH",
  "amount": "0.5",
  "source": {
    "type": "WALLET_POOL",
```

[Copy](#)

```
"id": "<poolId>"
},
"destination": {
  "type": "ONE_TIME_ADDRESS",
  "oneTimeAddress": {
    "address": "0xRecipientAddress"
  }
}
```

The create response returns the transaction ID and status only; it does not report which vault account was selected. To see that, retrieve the transaction with GET `/v1/transactions/{txId}` or wait for its webhook. On the retrieved record, source has been rewritten to the vault account Fireblocks selected, so `source.id` is the vault account that sent, and the pool appears in `source.tags`. The pool ID is also written onto the transaction record as `extraParameters.walletPoolId`, which is the field to reconcile against in transaction exports and webhooks.

Finding your pools and their members later

There is no separate pools endpoint. Use the tags and vaults endpoints:

- GET `/v1/tags?type=WALLET_POOL` lists every pool in the workspace with its ID and label.
- GET `/v1/vault/accounts_paged?includeTagIds=<poolId>` lists the vault accounts currently in a pool.

How it works

When you submit a transaction using a Wallet Pool as the source, Fireblocks:

1. Identifies all vault accounts in the pool that hold the requested asset, with enough available balance for that one vault account to cover the full transaction amount on its own. If no vault account qualifies, the transaction is rejected.
2. Narrows those candidates to the vault accounts that also hold enough of the base asset to cover the transaction's gas fee. This is a preference rather than a requirement: if no vault account in the pool can cover the fee, all funded vault accounts stay in play and the transaction still proceeds. Fireblocks skips this step where it cannot estimate a fee for the chain.
3. Queries Account Traffic Control (ATC) for the current health state of the remaining candidates. Vault accounts operating normally are preferred, then those showing early signs of congestion, and those with stuck transactions only as a last resort.

4. Selects a vault account from the best available group by round-robin. A transaction is never rejected for lack of a healthy vault account; if every vault account in the pool is congested, Fireblocks still sends through the best one available.
5. If ATC is unavailable, we fall back to balance scoring: the vault account with the fewest locked transactions and the highest available balance is selected. This selection is deterministic rather than rotating, so it returns the same vault account until balances change.

Renaming or deleting a pool

Renaming and deleting a pool both require quorum approval. A pool cannot be deleted while it still contains vault accounts. Remove all vault accounts first, then delete the pool.

Vault account membership and approval flows

The following actions require quorum approval from authorized signers:

Action	Approval required
Create pool	No
Add vault account to pool	Yes
Remove vault account from pool	Yes
Rename pool	Yes
Delete pool	Yes (pool must be empty first)

Approval requests appear in the Fireblocks mobile app and Console under **Pending Approvals**, the same as all other protected tag operations.

Monitoring your pool

To see the health and composition of a pool, open any vault account in the **Wallet Pools** widget, select the **more actions menu** (:) next to the pool, and select **Vault breakdown**.

For transaction-level detail, each transaction record shows both the pool and the resolved vault account. To build a distribution picture over time, retrieve your transactions and group them by `extraParameters.walletPoolId`, the field that marks a transaction as pool-routed.

Considerations and limitations

- **EVM chains only for health-aware routing.** On non-EVM chains, vault account selection falls back to balance scoring: the vault account with the fewest locked transactions and the highest available balance. This is deterministic rather than rotating, so it returns the same vault account until balances change. On a non-EVM chain, a pool therefore gives you a single logical source and a single Policy target, but not congestion avoidance or address rotation. Full health-aware routing for non-EVM chains will be added in a future release.
- **A vault account can belong to multiple pools.** If a vault account is shared across pools, its health state is shared as well. High transaction volume from one pool can affect vault account health as seen by another pool.
- **Vault account removal resets routing.** Removing a vault account recalculates all positions and resets the round-robin pointer to the first vault account.
- **Not a strict sequence.** Round-robin distributes evenly over time, but is not a strict sequence. Do not build reconciliation logic that expects a fixed repeating order.
- **No aggregated balance.** Balances are not pooled. A pool distributes transactions, not funds. Each transaction must be covered in full by a single vault account in the pool. A pool of ten vault accounts holding 1 ETH each cannot send a 5 ETH transaction; it is rejected for insufficient balance even though the pool holds 10 ETH in total. Adapt individual vault account balances to your largest expected transaction, not to your aggregate volume.

Troubleshooting

Transaction rejected: insufficient balance

None of the vault accounts in the pool hold enough of the requested asset to cover the transaction amount. Top up vault account balances or reduce the transaction amount. Remember that balances are not pooled, so one vault account must cover the full amount on its own.

Transaction rejected: asset not found in pool

None of the vault accounts in the pool hold the requested asset. Verify that the vault accounts in your pool are set up with the correct asset and try again.

Transaction sent from a vault account that could not cover the gas fee

Fireblocks prefers vault accounts that can cover the transaction's gas fee, but if no vault account in the pool can, it still sends from the best available one, and the transaction can then stall or fail. Keep every vault account in the pool funded with the native asset of its chain.

I don't know which vault account sent a transaction

Check the transaction detail in the Console, or retrieve the transaction over the API with GET `/v1/transactions/{txId}`. On the retrieved record, `source.id` and `source.name` are the vault account that sent, and the Console activity panel shows the same vault account name.

Related articles

- [Account Traffic Control \(/hc/en-us/articles/27129539023004\)](/hc/en-us/articles/27129539023004)
- [Protected Tags \(/hc/en-us/articles/27718237234972\)](/hc/en-us/articles/27718237234972)
- [Policies \(/hc/en-us/articles/19156998685980\)](/hc/en-us/articles/19156998685980)
- [Manage Vault Accounts \(/hc/en-us/articles/26075380927132\)](/hc/en-us/articles/26075380927132)
- [Gasless Transactions \(/hc/en-us/articles/19948199000092\)](/hc/en-us/articles/19948199000092)

Related articles

-  [Wallet Pool Tags](#)
-  [Use tags in automation rules](#)
-  [Protected Tags](#)
-  [Staking ADA using Raw Signing](#)
-  [About Embedded Wallets](#)
-  [Manage asset wallets](#)
-  [Embedded Wallets Takeover Option](#)
-  [Interacting with Web3 on Fireblocks](#)